FRONT MATTER

Abstract

Personal finance management is a challenge for students and young professionals who wish to control their spending but find conventional budgeting tools dull, abstract and easy to abandon. Existing applications focus heavily on recording expenses after the fact and rarely reward disciplined saving behaviour. **5thSave** addresses this gap by reframing personal budgeting as a goal-oriented, gamified activity.

In 5thSave a user defines a savings goal with a monetary *limit* (for example, “keep my weekly spending under ₹5,000”) and a time window. Every purchase is made through a built-in shopping simulator and is linked to a goal. Rather than measuring how much was spent, the application computes a **saver score** — the percentage of the limit that remains unspent — and grants virtual reward vouchers at the 25%, 50%, 75% and 100% milestones. A peer-circle and leaderboard subsystem adds a social dimension, letting users compare saver scores with friends invited through expiring share links. A “time skip” feature simulates the passage of days, weeks or months so that a savings window can be exercised realistically within a short demonstration session.

The application follows a three-tier architecture. The front end is a single-page application built with **React 19**, **Vite**, **Axios** and **Recharts**; the back end is a REST API built with **Node.js** and **Express 5**; and data is persisted in **PostgreSQL**. Authentication uses **JWT** with **bcrypt** password hashing, and money-critical operations such as checkout use database transactions to guarantee integrity. The backend exposes **29 REST endpoints** covering authentication, goals, cart and orders, the ledger, rewards, and circles.

The system was verified with a self-contained shell-based smoke test suite that boots its own server and exercises every route, including negative cases (invalid sign of a transaction, cross-user access attempts, duplicate reward guards). Results confirm that the saver-score engine, reward ledger, net-spend arithmetic, timeline logic and social features behave consistently. The report concludes with a discussion of limitations and a roadmap for future enhancements such as real bank integrations, cloud deployment and richer analytics.

Keywords: personal finance, budgeting, gamification, savings goals, REST API, React, Express, PostgreSQL, JWT, leaderboard.

FRONT MATTER

Table of Contents

1 Introduction

1.1 Background and motivation · 1.2 Problem statement · 1.3 Objectives · 1.4 Scope and limitations · 1.5 Organisation of the report

2 System Analysis

2.1 Existing systems · 2.2 Proposed system · 2.3 Functional requirements · 2.4 Non-functional requirements · 2.5 Feasibility study

3 System Design

3.1 Architecture overview · 3.2 Module decomposition · 3.3 Core algorithms and business rules · 3.4 User interface design

4 Database Design

4.1 Entity–relationship model · 4.2 Table specifications · 4.3 Indexing and integrity

5 Implementation

5.1 Technology stack · 5.2 Backend implementation · 5.3 Frontend implementation · 5.4 REST API reference

6 Testing and Validation
6.1 Testing strategy · 6.2 Test cases and results · 6.3 Observed defects and fixes
7 Results and Discussion
8 Conclusion and Future Scope
— References
— Appendix A — Setup and Run Instructions
— Appendix B — Database Schema (SQL)

CHAPTER 1

Introduction

1.1 Background and motivation

Budgeting is one of the most commonly recommended yet least consistently practised personal habits. The reason is rarely a lack of information; modern users have instant access to bank statements, UPI histories and card notifications. The real problem is that budgeting tools treat spending as an accounting exercise rather than a behavioural one. They show what has already been spent but provide no positive reinforcement for restraint, and they rarely create any sense of progress or achievement.

Behavioural economics repeatedly shows that immediate, visible rewards and social comparison are far stronger motivators than abstract long-term benefit. **Gamification** — the application of game-design elements such as points, levels, milestones and leaderboards to non-game contexts — has proven effective in fitness, learning and productivity. 5thSave applies the same principles to saving money. Instead of asking "how much did I spend?", it asks "how much did I *keep*?" and turns every saved rupee into measurable, visible progress.

1.2 Problem statement

The problem addressed by this project can be stated as follows:

Problem statement. Design and implement a web application that helps users set explicit savings limits, simulate real-world spending against those limits, objectively compute how much has been saved, and motivate continued discipline through milestone-based rewards and social comparison — while keeping the underlying financial records accurate, auditable and secure.

Conventional expense trackers fail on three counts: (i) they reward nothing, so the incentive to keep tracking fades; (ii) they measure expenditure, not savings, so the user never sees a positive score to defend; and (iii) they are typically single-user and offer no social accountability. 5thSave is designed to close all three gaps.

1.3 Objectives

The specific objectives of the project are:

1. To provide a secure, authenticated platform on which each user can create multiple **savings goals**, each defined by a monetary limit and a time window.
2. To implement a **shopping simulator** through which purchases are recorded and attributed to a specific goal, producing an accurate transaction ledger.
3. To compute a transparent **saver score** — the percentage of the limit that has been retained — and to award virtual reward vouchers at the 25%, 50%, 75% and 100% checkpoints.
4. To add a **social layer** consisting of peer circles, secure invitation links and a savings leaderboard that ranks members by their best saver score.
5. To support **time simulation** (day / week / month skips) so that goal windows can be exercised without waiting in real time.

6. To present clear **analytics and reports** — spending by category, saving by goal, and overall saved-versus-spent figures.
7. To ensure data **integrity and security** through parameterised queries, hashed passwords, signed tokens and atomic transactions at checkout.

1.4 Scope and limitations

5thSave is a **simulation-based** budgeting trainer. Purchases are made from an internal product catalogue rather than from real merchants, and rewards are virtual vouchers rather than real credit. This is a deliberate design decision: it keeps the project self-contained and avoids handling real money, while still exercising the complete logic of goal tracking, scoring and reward issuance. The system is intended as a functional prototype and learning tool, not as a production financial product.

Within this scope the application supports multiple users, multiple concurrent goals per user, full order history, per-goal and per-circle views, and a persistent relational data model.

1.5 Organisation of the report

Chapter 2 analyses existing budgeting tools and derives the requirements of the proposed system. **Chapter 3** presents the system architecture, module decomposition and core algorithms. **Chapter 4** documents the database design in detail. **Chapter 5** describes the implementation, technology choices and the complete REST API. **Chapter 6** covers the testing strategy and results. **Chapter 7** discusses outcomes, and **Chapter 8** concludes with future scope. Setup instructions and the schema SQL are provided in the appendices.

CHAPTER 2

System Analysis

2.1 Existing systems and their limitations

Personal budgeting is served today by three broad categories of tools. Their strengths and shortcomings are summarised below.

Category	Representative tools	Limitations relevant to this project
Manual method	Spreadsheets, paper ledgers, notes apps	Fully manual entry, error-prone, no validation, no reinforcement, easy to abandon, no social layer.
Expense trackers	Money View, Walnut and similar apps	Record spending after the fact; measure expense rather than savings; no positive score to defend; limited gamification; typically single-user.
Bank / budget apps	Bank spend analytics, envelope-budgeting apps	Read-only analytics or rigid categories; little behavioural incentive; social comparison absent; often tied to a specific bank or account.

The common weakness is behavioural: none of these tools turn restraint into something the user actively wants to protect. They answer “where did the money go?” but not “how well am I doing, and can I do better?”

2.2 Proposed system

5thSave reframes the problem around **saving as a score to be defended**. Its distinguishing features are:

- **Savings-first metric.** The primary measure is the saver score, (limit − spent) ÷ limit × 100, clamped at zero. Users track what they keep, not what they spend.
- **Goal-linked spending.** Every purchase is charged to a chosen goal, so each goal has an independent, honest ledger.

- **Milestone rewards.** Reaching 25%, 50%, 75% or 100% savings at the end of a goal unlocks a named voucher — a concrete, immediate payoff.
- **Net-spend correctness.** Refunds and positive adjustments reduce effective spending, so a refund never punishes the user's score.
- **Time simulation.** Day / week / month skips let a goal window elapse on demand, and a goal that crosses its end date auto-completes and settles rewards.
- **Social accountability.** Peer circles with expiry-limited invite links and a leaderboard ranked by best saver score.
- **Reporting.** Category-wise spending, per-goal saved-versus-spent tables and charts, and an overall savings summary.

2.3 Functional requirements

ID	Requirement	Description
FR-01	User registration & login	Users register with a unique username, e-mail and password; passwords are hashed; login issues a signed JWT valid for 24 hours.
FR-02	Session management	The client stores the token, rehydrates the profile on reload, and automatically signs the user out on an expired or invalid token.
FR-03	Create & manage goals	Create a goal with a positive limit, a start date and an end date (end must be after start); list, finish, or delete goals individually or in bulk.
FR-04	Browse catalogue	View products by category (Food, Clothes, Electronics, Accessories) with per-category counts.
FR-05	Cart	Add items (quantity-upsert), view cart totals, and remove items.
FR-06	Direct buy & checkout	Buy a single product directly, or check out the whole cart, charging the total to a selected active goal and writing a spend to the ledger atomically.
FR-07	Transaction ledger	Append-only record of spends, refunds and adjustments; sign must match the transaction type.
FR-08	Saver score & rewards	Compute saver score from the goal limit and net spending; award each milestone voucher at most once per goal.
FR-09	Finish a goal	Manually end an active goal, settling its rewards and reporting the final saver score.
FR-10	Time skip	Advance a goal's start date by 1, 7 or 30 days; auto-complete and settle if the window has elapsed.
FR-11	Order history & report	Retrieve all purchased line items with category; aggregate spending by category and saving by goal.
FR-12	Peer circles	Create and join circles, leave a circle, and delete a circle (creator only).
FR-13	Invitations	Generate a multi-use, 7-day invite token; preview an invite link publicly; join via token.
FR-14	Leaderboard	Rank circle members by their best saver score across goals; members only.

2.4 Non-functional requirements

- **Security.** Passwords hashed with bcrypt (cost factor 10); stateless JWT authentication; all data access scoped to the authenticated user; SQL through parameterised queries to prevent injection.

- **Integrity.** Checkout runs inside a database transaction (BEGIN / COMMIT / ROLLBACK) so an order, its line items, its ledger entries and the cart clear either all succeed or all fail.
- **Correctness.** Reward issuance is idempotent — an existence guard prevents duplicate vouchers for the same checkpoint.
- **Usability.** A single-page interface with four primary views, inline validation, toasts for feedback, and plain-language messaging that avoids financial jargon.
- **Maintainability.** Clear separation of concerns (routes, helpers, data access), a single shared database pool, and environment-driven configuration.
- **Portability.** Runs on any environment with Node.js and PostgreSQL; the front end builds to static assets.
- **Testability.** A self-contained smoke-test script that boots its own server and cleans up the data it creates.

2.5 Feasibility study

Dimension	Assessment
Technical	The stack (React, Node/Express, PostgreSQL) is mature, well-documented and widely available. All components are open source and installable without specialised hardware.
Economic	Development uses free and open-source tooling and can run on a single developer machine or a low-cost cloud instance. No licensing costs are incurred.
Operational	The interface requires no training: users recognise goals, carts and leaderboards from everyday apps. Deployment is a Node process plus a database.
Schedule	The project was delivered incrementally, with the core engine and front end first, followed by cart/orders, reward and reporting refinements, and finally correctness fixes.

CHAPTER 3

System Design

3.1 Architecture overview

5thSave follows a classic **three-tier architecture**. The presentation tier is a React single-page application; the application tier is an Express REST API that holds all business logic; and the data tier is a PostgreSQL relational database. The tiers communicate over HTTP with JSON payloads, and the application tier is the only component that touches the database.

Tier	Component	Responsibility
Presentation	React 19 SPA — Vite, Axios, Recharts	Four views (Dashboard, Buy, Report, Circles), client-side state, charts, and a central Axios interceptor that handles session expiry.
Application	Node.js / Express 5 REST API	29 endpoints, JWT authentication, and all business rules: saver score, reward engine, time-skip, atomic checkout and the leaderboard query.
Data	PostgreSQL	Eleven relational tables, foreign keys with cascade, check constraints and indexes on the hot access paths.

The lifecycle of a request is straightforward: a user action in a React view is sent by Axios with a bearer token; the Express route runs the authentication middleware and then the business logic and SQL; the resulting JSON updates the client state and shows a toast confirmation.

3.2 Module decomposition

Module	Responsibilities	Key endpoints
Authentication	Registration with validation and hashing, login, token issuing, profile retrieval, session-expiry handling.	/auth/register /auth/login /auth/me
Goals	Create goals with date validation, list with aggregated spend, finish, delete one or all, clear rewards.	/goals /goals/:id/end /goals/:id/rewards
Shop & catalogue	Serve products and categories; two purchase paths (direct buy and cart checkout).	/products /shop/buy /shop/checkout
Cart & orders	Cart upsert, listing and removal; order and line-item persistence; order history for reporting.	/cart /orders
Ledger	Append-only spends, refunds and adjustments with sign validation.	/transactions
Rewards engine	Compute saver score, award milestone vouchers idempotently, list a user's rewards.	/rewards (internal helper)
Time simulation	Advance a goal's clock and detect window expiry with automatic completion.	/goals/:id/skip-time
Social circles	Create, join, leave and delete circles; membership and creator transfer; leaderboard.	/circles /leaderboard/:id
Invitations	Generate and resolve expiring share tokens; public invite preview.	/circles/invite /circles/join-invite

3.3 Core algorithms and business rules

3.3.1 Saver score

The heart of the application is the saver score. Spending is recorded in the ledger as negative amounts and refunds or adjustments as positive amounts, so the net outflow for a goal is the negation of the sum of ledger amounts. The score is then:

```
net_spent  = max(0, -SUM(amount) for the goal)     -- refunds offset spends
saver_score = max(0, (amount_limit - net_spent) / amount_limit * 100)
```

Using the *net* sum (rather than summing only negative rows) is important: a refund must increase the score, never decrease it. The score is clamped to a minimum of zero so that overspending reports 0% rather than a negative percentage.

3.3.2 Reward engine

Rewards are granted only when a goal is settled — either explicitly finished by the user or completed automatically when its window elapses. This prevents spending-based point farming. At settlement the engine walks the checkpoints in ascending order and, for each checkpoint satisfied by the score, inserts a reward row only if one does not already exist.

Checkpoint	Requirement	Reward voucher
25%	Saver score ≥ 25	Flipkart Voucher
50%	Saver score ≥ 50	Amazon Giftcard
75%	Saver score ≥ 75	Makemytrip Voucher

Checkpoint	Requirement	Reward voucher
100%	Saver score = 100	Ultimate Saver Reward

Table 3.1 — Milestone reward mapping. Checkpoints are cumulative: scoring 80% satisfies the 25%, 50% and 75% levels.

3.3.3 Checkout as an atomic transaction

Cart checkout is the most integrity-critical operation. It performs the following steps inside a single database transaction, so any failure rolls back every change:

1. Verify the goal exists, belongs to the buyer, and is still active.
2. Read the cart line items and compute the order total.
3. Insert an orders row.
4. For each item: insert an order_items row and a negative spend entry in the ledger.
5. Clear the user's cart.
6. Commit, then advance the goal's clock by one day (a purchase "consumes" a day of the window) and settle rewards if the window has now elapsed.

3.3.4 Time skip and window expiry

The time-skip helper shifts a goal's start date forward by 1, 7 or 30 days. After the shift, if the (moved) start date has reached or passed the end date, the goal is marked completed automatically and rewards are settled in the same request. This models the natural expiry of a savings window without requiring real waiting, and it is the same code path used by the automatic one-day advance at checkout.

3.3.5 Leaderboard ranking

The leaderboard aggregates each member's goals into per-goal saver percentages and then takes the best percentage per member, ordering descending with members who have no goals placed last. A sub-query computes each goal's percentage from its limit and ledger sum, and the outer query groups by member.

3.4 User interface design

The interface is a single-page application with a persistent top bar and a four-view navigation. Each view has its own accent theme inspired by the colours of Indian currency notes, which gives the app a distinct identity and helps users orient themselves by colour.

View	Purpose	Theme accent
Dashboard	Portfolio summary, goal creation and management, milestone tracker, saved-versus-left pie chart and budget-by-goal bar chart.	Stone grey (#4F6B6A)
Buy	Categorised product grid, goal selector, cart with running total, checkout, receipt and a live "savings spotlight" after purchase.	Bright yellow (#C97A1B)
Report	Total-saved summary, spending-by-category pie and table, saving-by-goal bar chart and table.	Report blue (#1D4ED8)
Circles	Circle list with create / share / leave / delete actions, invite sheet, and the top-savers leaderboard.	Lavender (#3B3566)

Interaction feedback is provided through transient toast messages (for example, "Bought! You saved 62%") and through modal-like sheets for the receipt and the invite-sharing menu. After a purchase, a *savings spotlight* panel shows the deduction, the remaining saver score, which milestone levels are still secured, and the maximum additional spend that would preserve the current level — a deliberately plain-language hint.

CHAPTER 4

Database Design

4.1 Entity–relationship model

The schema is normalised and centred on the users and savings_goals entities. The transaction ledger is append-only, and every child record cascades from its parent so that deleting a user or a goal leaves no orphans.

Table	Key columns	Role and relationships
users	id, username, email, password_hash	Credentials and profile. One user owns many goals, orders and rewards; many-to-many with circles through circle_members.
savings_goals	id, user_id, amount_limit, start_date, end_date, status	Target limits and windows. Belongs to a user; has many ledger entries and rewards.
transaction_ledger	id, goal_id, amount, transaction_type, description, timestamp	Append-only money movements. Negative for a spend, positive for a refund or adjustment.
rewards	id, user_id, goal_id, checkpoint_percentage, reward_item, earned_at	Vouchers earned per checkpoint on a goal (25 / 50 / 75 / 100).
products	id, name, price, category	Shopping catalogue (21 seeded items across four categories).
cart_items	id, user_id, product_id, quantity, added_at	Per-user shopping cart; unique on (user_id, product_id).
orders	id, user_id, goal_id, total_amount, order_date	Purchase headers (receipts) linked to the user and the charged goal.
order_items	id, order_id, product_id, price_at_purchase, quantity	Order line items, recording the price at the time of purchase.
peer_circles	id, circle_name, created_by, created_at	Social groups; the creator is the only member who may delete the circle.
circle_members	circle_id, user_id, joined_at	Membership mapping with a composite primary key.
invites	id, circle_id, token, invited_by, expires_at	Expiring, multi-use invite links belonging to a circle.

4.2 Table specifications

Table	Notable constraints
users	Unique username and e-mail; non-null password hash.
savings_goals	Foreign key to users (cascade); status defaults to *active*.
transaction_ledger	Foreign key to savings_goals (cascade); indexed on goal_id.
rewards	Checkpoint restricted by CHECK to the four milestone values; indexed on user_id.
products	Unique product name; price must be non-negative.
cart_items	Unique (user_id, product_id); quantity must be greater than zero.
orders / order_items	Cascade from user and order respectively; quantity greater than zero.

Table	Notable constraints
peer_circles / circle_members	Composite primary key on membership; created_by set to null if the creator is deleted.
invites	Unique token; invited_by set to null if the inviter is deleted.

4.3 Indexing and integrity

Referential integrity is enforced with foreign keys, most of which specify ON DELETE CASCADE so that dependent rows are removed with their parent. The invites.invited_by and peer_circles.created_by columns use ON DELETE SET NULL so that deleting a user does not destroy a circle or an invite trail. Domain constraints are expressed with CHECK clauses: reward checkpoints are restricted to the four milestone values, and cart and order quantities must be positive. Unique constraints protect username, email, product name, invite token, and the (user_id, product_id) cart pair.

Three indexes target the most frequent access paths: one on the ledger by goal, one on goals by user, and one on rewards by user. The follow-up migration is idempotent (using IF NOT EXISTS and ON CONFLICT DO NOTHING), so it can be re-run safely, and it also back-fills legacy circles with a creator.

CHAPTER 5

Implementation

5.1 Technology stack

Layer	Technology	Role in the project
Frontend	React 19 · Vite 8 · Axios · Recharts	SPA component model and state, dev server and production build, HTTP client with a response interceptor, and pie and bar charts.
Frontend	React Router · Oxlint	Routing support for invite URLs and static linting of the source.
Backend	Node.js (v20) · Express 5 · pg	Server runtime, routing and middleware, and the PostgreSQL connection pool and queries.
Backend	jsonwebtoken · bcryptjs · cors · dotenv	Stateless authentication tokens, password hashing (cost factor 10), cross-origin access and configuration loading.
Database	PostgreSQL	Relational persistence with transactions, constraints and cascades.
Testing	Bash · curl · psql	A self-contained smoke suite exercising every route.

Table 5.1 — Technology stack. Version ranges reflect the project's declared dependencies.

5.2 Backend implementation

5.2.1 Data access layer

A single connection pool is created once and shared across the application. A thin module exposes a query helper for ordinary statements and a getClient helper for operations that need a dedicated client for a transaction.

```
const pool = new Pool({ connectionString: process.env.DATABASE_URL });
module.exports = {
  query: (text, params) => pool.query(text, params),
  getClient: () => pool.connect(),
};
```

5.2.2 Configuration and startup guards

Environment variables are loaded explicitly from backend/.env, pinned to that path regardless of the working directory, because a second .env at the repository root carries a different JWT_SECRET and would silently invalidate issued tokens. The server refuses to start if JWT_SECRET is missing, failing fast rather than issuing insecure tokens.

```
require('dotenv').config({ path: path.join(__dirname, '.env') });
if (!process.env.JWT_SECRET) {
  console.error('FATAL: JWT_SECRET is not set...');
  process.exit(1);
}
```

5.2.3 Authentication middleware

Protected routes pass through a middleware that extracts the bearer token, verifies it with the secret, and attaches the decoded user to the request. Missing or expired tokens yield a clear 401 with a human-readable message that the client uses to force re-authentication.

```
const authenticateToken = (req, res, next) => {
  const token = (req.headers['authorization'] || '').split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Not signed in...' });
  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) return res.status(401).json({ error: 'Session expired...' });
    req.user = user;
    next();
  });
};
```

5.2.4 Reward and time-skip helpers

Two shared helper functions hold the financial core. checkAndAwardRewards computes the net spend and saver score and awards each satisfied checkpoint exactly once. advanceClock moves a goal’s start date and, if the window has elapsed, completes the goal and settles rewards — a path reused by both the explicit time-skip endpoint and the automatic one-day advance at checkout.

```
// Net outflow: spends are negative, refunds offset them.
const spendingRes = await db.query(
  'SELECT COALESCE(-SUM(amount), 0) AS spent FROM transaction_ledger WHERE goal_id = $1',
  [goalId]
);
totalSpent = Math.max(0, parseFloat(spendingRes.rows[0].spent || 0));
saverScore = Math.max(0, ((limit - totalSpent) / limit) * 100);
```

5.2.5 Validation strategy

Every mutating route validates its inputs before touching the database: positive integers for identifiers and quantities, a positive limit for goals, a valid date ordering, and a transaction sign that matches its type. Because all identifiers are also checked against the authenticated user, cross-user access attempts return 403 rather than leaking data.

5.3 Frontend implementation

The client is a single React component tree with a shared top bar and four conditional views. State is held with React hooks, and derived data (portfolio summary, report figures, chart series) is computed with useMemo so that charts and tables stay consistent with the latest server response.

5.3.1 Session handling

On load, if a token is present the client rehydrates the profile from /auth/me and fetches all collections. A response interceptor watches for 401 responses and, unless the request was itself a login or register attempt, clears the token and returns the user to the sign-in screen — so an expired session never leaves the UI in a silently broken state.

5.3.2 The savings spotlight

After a purchase, the client compares the goal's pre- and post-purchase spending to compute the new saver score. It then displays which milestone levels remain secured and the maximum cumulative spend that would preserve the highest secured level. This is intentionally framed as a spending cap ("keep spending under ₹X to keep the Amazon Giftcard") rather than as an instruction to spend more, because additional spending can only lower the score.

5.3.3 Reporting and visualisation

Order history supplies category totals, which drive a pie chart and a ranked table; goals supply the saved-versus-spent bar chart and table. Recharts renders all charts, and currency is formatted throughout with the Indian numbering convention.

5.4 REST API reference

The backend exposes **29 endpoints**. Endpoints marked "Yes" in the Auth column require a valid bearer token. All request and response bodies are JSON.

Authentication and profile

Method	Path	Auth	Description
POST	/auth/register	—	Create an account. Validates e-mail format and password length; hashes the password; 409 on duplicate.
POST	/auth/login	—	Verify credentials and return a 24-hour JWT plus basic profile.
GET	/auth/me	Yes	Return the signed-in user's profile.

Goals

Method	Path	Auth	Description
POST	/goals	Yes	Create a goal (positive limit, valid dates, start before end).
GET	/goals	Yes	List the user's goals with aggregated total spend and transaction count.
POST	/goals/:id/end	Yes	Finish an active goal, settle rewards, return the saver score.
POST	/goals/:id/skip-time	Yes	Advance the goal by day / week / month; auto-complete on expiry.
DELETE	/goals/:id	Yes	Delete one goal (ledger, orders and rewards cascade).
DELETE	/goals/:id/rewards	Yes	Clear the rewards of one goal without deleting it.
DELETE	/goals	Yes	Delete all of the user's goals.

Shop, cart, orders and ledger

Method	Path	Auth	Description
GET	/products	—	List products, optionally filtered by ?category=.
GET	/products/categories	—	List categories with product counts.
POST	/shop/buy	Yes	Buy one product directly against a goal.
POST	/shop/checkout	Yes	Atomic checkout of the whole cart, then auto-advance one day.
POST	/cart	Yes	Add to cart (quantity upsert).
GET	/cart	Yes	List the cart with line totals.
DELETE	/cart/:id	Yes	Remove a cart line.
GET	/orders	Yes	Full order history with product names and categories.
POST	/transactions	Yes	Append a spend, refund or adjustment with sign validation.

Rewards

Method	Path	Auth	Description
GET	/rewards	Yes	List the user’s earned reward vouchers in order.

Circles, invitations and leaderboard

Method	Path	Auth	Description
GET	/circles	Yes	List the user’s circles with member counts and creator flag.
POST	/circles	Yes	Create a circle; the creator is auto-joined.
POST	/circles/join	Yes	Join a circle by id.
DELETE	/circles/:id/leave	Yes	Leave a circle; transfer creator or delete if empty.
DELETE	/circles/:id	Yes	Delete a circle (creator only).
POST	/circles/invite	Yes	Create a multi-use 7-day invite token (members only).
POST	/circles/join-invite	Yes	Join a circle using an invite token.
GET	/circles/invite/:token	—	Public preview of a valid invite (circle name, member count, inviter).
GET	/leaderboard/:circleId	Yes	Rank circle members by best saver score (members only).

CHAPTER 6

Testing and Validation

6.1 Testing strategy

Testing in 5thSave is deliberately lightweight but thorough on the parts that carry financial logic. The project ships a self-contained smoke-test script (backend/tests/smoke.sh) that boots its own server on an isolated port, registers throw-away users tagged with a random suffix, exercises every route in sequence, and then

kills the server and deletes the test users on exit. This keeps the test hermetic: it does not depend on any pre-existing data and does not pollute the development database.

The suite combines **happy-path** checks (registration, login, goal creation, purchase, checkout, circle lifecycle) with **negative-path** checks that assert the system rejects invalid input correctly — a wrong password, a missing token, a negative goal limit, a cross-user goal reference, a positive "spend", and a duplicate attempt to finish a goal. A dedicated case verifies the net-spend arithmetic: spending 800 and refunding 500 on a ₹1,000 goal must yield a saver score of 70, not 20.

6.2 Test cases and results

ID	Case	Expected result	Status
T-01	Register a new user with valid data	201 with new user id	Pass
T-02	Login with the registered credentials	200 with a JWT	Pass
T-03	Login with a wrong password	401 "Invalid credentials"	Pass
T-04	Access a protected route without a token	401 "Not signed in"	Pass
T-05	Create a goal with a negative limit	400 referencing the limit	Pass
T-06	List goals for a fresh user	Total spend = 0	Pass
T-07	Add a valid product to the cart	201, quantity 1	Pass
T-08	Add a non-existent product	404 "Product not found"	Pass
T-09	Buy a product against another user's goal	403 "not yours"	Pass
T-10	Checkout an empty cart	400 "Cart is empty"	Pass
T-11	Post a positive amount as a "spend"	400 "must be negative"	Pass
T-12	Spend 800 then refund 500 on a ₹1,000 goal, then finish	Saver score = 70	Pass
T-13	Skip time by an invalid interval	400 "Invalid interval"	Pass
T-14	Finish the same goal twice	400 "already finished"	Pass
T-15	Create a circle and invite a second user	Token issued; second user joins	Pass
T-16	Open the leaderboard of a non-member circle	403 "must be a member"	Pass
T-17	Leave a circle	200 "Left circle."	Pass
T-18	Clear rewards for a goal, then delete it and its circle	Success messages; data removed	Pass

Table 6.1 — Representative smoke-test cases. The suite prints a pass/fail summary and exits non-zero if any case fails.

Automated verification. Running the suite produces a summary line of the form --- PASS=43 FAIL=0 ---. The script is wired to npm test in the backend package and requires only a running PostgreSQL instance and a populated backend/.env.

6.3 Observed defects and fixes

Development surfaced several subtle issues that were corrected before the final revision. They are recorded here because each illustrates a correctness or usability lesson embedded in the code.

Issue	Resolution
Refunds were ignored when summing only negative ledger rows, so a refund lowered the effective savings.	Switched the score query to a net sum so refunds offset spends.
A positive amount labelled “spend” behaved like a refund, silently corrupting scores.	Added sign validation: spends must be negative, refunds positive, and zero is rejected.
Tokens were intermittently invalidated because a root .env with a different secret was sometimes loaded.	Pinned dotenv to backend/.env and abort startup when the secret is missing.
Duplicate milestone vouchers could be issued on repeated settlement calls.	Added an existence guard before each reward insert, plus an “already finished” guard on the end route.
The post-purchase hint told users they could “spend more to reach a higher level”, which is impossible.	Reframed the hint as the maximum cumulative spend that preserves the currently secured level.
Legacy circles had no creator, so nobody could manage them.	The migration back-fills the creator from the longest-standing member.

CHAPTER 7

Results and Discussion

The completed application delivers a working, end-to-end savings platform. A user can register, set one or more savings goals, browse a catalogue of 21 products across four categories, add items to a cart, and check out against a chosen goal. Each purchase is recorded twice — once as an order line and once as an append-only ledger entry — so the history is both human-readable and auditable. The saver score updates immediately, the dashboard and report views reflect the new position, and finishing a goal (manually or by letting its window elapse) settles the milestone rewards that the score has earned.

In summary, the finished system comprises **29 REST endpoints**, **eleven database tables**, a smoke suite of **43 checks**, **four milestone reward levels** and **four application views**.

7.1 What works well

- **A honest, defensible metric.** The saver score is transparent and, thanks to net-spend arithmetic, robust to refunds. Users can reason about it directly.
- **Integrity by construction.** Checkout is atomic; foreign keys cascade; rewards are idempotent. The data model makes several classes of bug structurally impossible.
- **Motivating design.** Milestones, vouchers, the post-purchase spotlight and the leaderboard give the user immediate, social and cumulative reasons to keep spending low.
- **Practical time simulation.** Because a goal window can be advanced on demand, the entire savings lifecycle can be demonstrated in minutes rather than weeks.
- **A cohesive interface.** The four currency-inspired themes and plain-language messaging make the app feel finished and approachable.

7.2 Limitations

- Spending is simulated through an internal catalogue; there is no integration with real bank or UPI transactions.
- Rewards are virtual and are not redeemable with external merchants.
- The front-end build is served separately from the API; production would require a reverse proxy or static hosting plus HTTPS.

- There is no e-mail verification, password reset, or administrative console.
- Search is presented in the interface but is not yet wired to a query endpoint.

7.3 Discussion

The most instructive part of the project was the interaction between behavioural design and correctness. Gamification only works if the underlying score is trustworthy; an appealing interface layered over arithmetic that mishandles refunds would actively mislead users. Conversely, a mathematically correct score that is never surfaced in a motivating way would be ignored. 5thSave treats the two as inseparable: the reward engine and the score are computed in one place, and the interface always reports the resulting position in plain language. This discipline — deciding exactly what is being measured, then making that measurement visible and rewarding — is arguably the central engineering lesson of the project.

CHAPTER 8

Conclusion and Future Scope

8.1 Conclusion

5thSave demonstrates that personal budgeting can be re-imagined as a goal-oriented, gamified activity. By measuring what the user *keeps* rather than what they *spend*, and by rewarding disciplined saving with milestone vouchers and social recognition, the application supplies the immediate feedback that conventional trackers lack. The system was designed with a clear three-tier architecture, implemented with a modern React and Express stack over PostgreSQL, secured with hashed passwords and signed tokens, and verified with an automated suite that exercises every route including its error paths. All stated objectives — secure accounts, multi-goal management, goal-linked purchasing, a transparent saver score, milestone rewards, peer circles with a leaderboard, and time simulation — were met and confirmed by testing.

8.2 Future scope

1. **Real financial integrations.** Connect to bank and UPI feeds (via account-aggregator APIs) so that actual transactions drive the ledger, turning the simulator into a genuine budgeting tool.
2. **Redeemable rewards.** Partner with merchants so that milestone vouchers have real value, or convert them into in-app currency.
3. **Cloud deployment and CI.** Containerise the API and database, host the front end on a CDN, and run the smoke suite automatically in a continuous-integration pipeline on every commit.
4. **Richer analytics.** Trend lines over time, category budgets, forecasting based on the simulated clock, and exportable PDF or CSV statements.
5. **Deeper gamification.** Streaks, badge collections, weekly challenges and a global leaderboard in addition to circle-level rankings.
6. **Account security hardening.** E-mail verification, password reset, refresh tokens, rate limiting and an administrative moderation panel.
7. **Mobile experience.** A progressive web app or native client reusing the same REST API.

The architecture already anticipates several of these extensions: the API is stateless and versionable, the ledger is append-only so it can absorb real transactions, and business rules such as the reward engine are isolated in reusable helpers.

BACK MATTER

References

1. React Documentation — https://react.dev (components, hooks, and state management).
2. Vite Documentation — https://vite.dev (development server and build).
3. Express.js Documentation — https://expressjs.com (routing and middleware).

4. Node.js Documentation — https://nodejs.org/docs.
5. PostgreSQL Documentation — https://www.postgresql.org/docs (transactions, constraints, indexes).
6. node-postgres (pg) Documentation — https://node-postgres.com (connection pooling and queries).
7. JSON Web Tokens Introduction — https://jwt.io/introduction.
8. bcrypt and bcryptjs — password-hashing algorithm and its JavaScript implementation.
9. Recharts Documentation — https://recharts.org (charting library for React).
10. Axios Documentation — https://axios-http.com (HTTP client and interceptors).
11. OWASP Application Security Verification Standard — secure authentication and input-validation guidance.
12. Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). *From game design elements to gamefulness: defining "gamification"*. MindTrek '11.

APPENDIX A

Setup and Run Instructions

Prerequisites

- Node.js (v20 or later) and npm
- PostgreSQL (a running server and a database)

1. Database

Create a database and apply the schema, then the follow-up migration (which is safe to re-run):

```
createdb 5thsave
psql "$DATABASE_URL" -f database/init.sql
psql "$DATABASE_URL" -f database/migrations/update_schema.sql
```

2. Backend

Create backend/.env with the connection string, the signing secret and the port. The secret must be identical anywhere it is loaded, or issued tokens will fail verification.

```
DATABASE_URL=postgres://user:password@localhost:5432/5thsave
JWT_SECRET=replace-with-a-long-random-secret
PORT=3000
```

```
cd backend
npm install
npm start        # API on http://localhost:3000
npm test         # smoke suite (needs PostgreSQL running)
```

3. Frontend

```
cd frontend
npm install
npm run dev      # Vite dev server
npm run build    # production build into dist/
```

The front end reads the API base URL from VITE_API_URL, defaulting to http://localhost:3000.

4. Project structure

```
5thsave/
  backend/          Express REST API
```

```
  index.js          29 endpoints, auth, reward and time helpers
  db.js             PostgreSQL connection pool
  tests/smoke.sh    self-contained smoke suite
database/
  init.sql          core schema (users, goals, ledger, rewards, circles)
  migrations/
    update_schema.sql   products, cart, orders, invites (idempotent)
frontend/           React 19 + Vite SPA
  src/App.jsx       four views, charts, cart, circles
README.md
```

APPENDIX B

Database Schema (SQL)

Core schema — init.sql

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE savings_goals (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    amount_limit DECIMAL(12, 2) NOT NULL,
    start_date TIMESTAMP WITH TIME ZONE NOT NULL,
    end_date TIMESTAMP WITH TIME ZONE NOT NULL,
    status VARCHAR(20) DEFAULT 'active',        -- active, completed, failed
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE transaction_ledger (
    id SERIAL PRIMARY KEY,
    goal_id INTEGER REFERENCES savings_goals(id) ON DELETE CASCADE,
    amount DECIMAL(12, 2) NOT NULL,             -- negative = spend, positive = refund
    transaction_type VARCHAR(20) NOT NULL,      -- 'spend', 'refund', 'adjustment'
    description TEXT,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE rewards (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    goal_id INTEGER REFERENCES savings_goals(id) ON DELETE CASCADE,
    checkpoint_percentage INTEGER NOT NULL CHECK (checkpoint_percentage IN (25, 50, 75, 100)),
    reward_item TEXT NOT NULL,
    earned_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE peer_circles (
    id SERIAL PRIMARY KEY,
    circle_name VARCHAR(100) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE circle_members (
    circle_id INTEGER REFERENCES peer_circles(id) ON DELETE CASCADE,
```

```
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    joined_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (circle_id, user_id)
);

CREATE INDEX idx_ledger_goal ON transaction_ledger(goal_id);
CREATE INDEX idx_goals_user  ON savings_goals(user_id);
CREATE INDEX idx_rewards_user ON rewards(user_id);
```

Follow-up schema — update_schema.sql (excerpt)

```
CREATE TABLE IF NOT EXISTS products ( ... );          -- catalogue, 21 seeded items

CREATE TABLE IF NOT EXISTS cart_items (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    product_id INTEGER REFERENCES products(id),
    quantity INTEGER DEFAULT 1 CHECK (quantity > 0),
    added_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (user_id, product_id)
);

CREATE TABLE IF NOT EXISTS orders (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    goal_id INTEGER REFERENCES savings_goals(id) ON DELETE CASCADE,
    total_amount DECIMAL(12, 2) NOT NULL,
    order_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS order_items (
    id SERIAL PRIMARY KEY,
    order_id INTEGER REFERENCES orders(id) ON DELETE CASCADE,
    product_id INTEGER REFERENCES products(id),
    price_at_purchase DECIMAL(12, 2) NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0)
);

CREATE TABLE IF NOT EXISTS invites (
    id SERIAL PRIMARY KEY,
    circle_id INTEGER REFERENCES peer_circles(id) ON DELETE CASCADE,
    token VARCHAR(64) UNIQUE NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    expires_at TIMESTAMP WITH TIME ZONE NOT NULL
);
ALTER TABLE invites ADD COLUMN IF NOT EXISTS invited_by INTEGER REFERENCES users(id) ON DELETE
SET NULL;
ALTER TABLE peer_circles ADD COLUMN IF NOT EXISTS created_by INTEGER REFERENCES users(id) ON
DELETE SET NULL;
```